

MemoryOS

Institutional Memory Engine

Technical Documentation, Testing Procedures & Reference
Guide

Authored by

Duane Cash

Principal Generative AI Engineer

GitHub: [dcash10181-gh](#)

Version 1.0.0 • May 2026

TABLE OF CONTENTS

1. Overview & Purpose

2. System Architecture

- 2.1 Data flow
- 2.2 Knowledge graph schema
- 2.3 MCP server & tools

3. Installation

- 3.1 Prerequisites
- 3.2 One-line installer
- 3.3 Manual install

4. Configuration

- 4.1 config.yaml reference
- 4.2 Environment variables

5. Connectors

- 5.1 Slack (Tier 1)
- 5.2 GitHub (Tier 2)
- 5.3 Jira (Tier 2)

6. Testing Procedures

- 6.1 Smoke test checklist
- 6.2 Neo4j browser verification
- 6.3 MCP tool testing via curl
- 6.4 Claude Desktop integration test

7. CLI Reference

8. Troubleshooting

9. Technical Concepts Explained

10. Roadmap

1. OVERVIEW & PURPOSE

What is MemoryOS?

MemoryOS is a self-hosted, open-source institutional memory engine that solves one of the most expensive invisible problems in enterprise engineering: **the loss of tacit knowledge**. Every day, companies lose context when employees leave, Slack threads expire, and Jira tickets close with no record of why a decision was made. MemoryOS captures that context passively and makes it queryable by any AI model via the Model Context Protocol (MCP).

The core problem it solves

- Engineers spend 19% of their week searching for information that already exists (McKinsey Global Institute)
- New hires take 3–9 months to reach full productivity — mostly rebuilding lost context
- When a senior engineer leaves, ~\$31.5M in institutional knowledge walks out (IDC)
- Existing tools (Confluence, Notion, SharePoint) are filing cabinets — they store but cannot reason
- AI tools like Claude and Copilot have powerful reasoning but zero knowledge of your company history

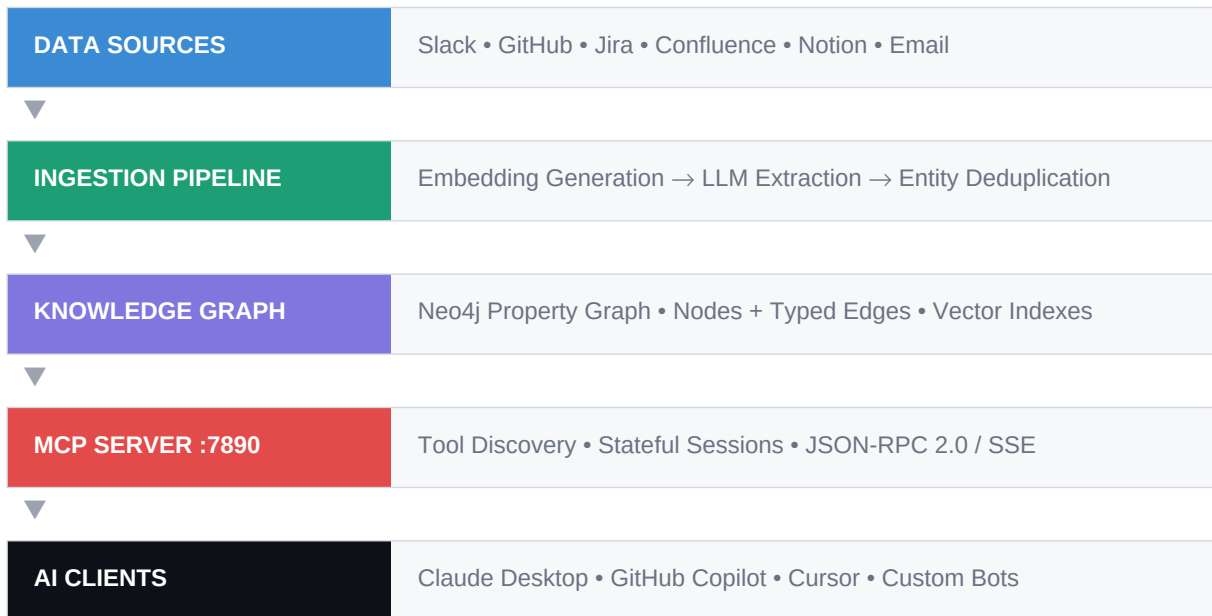
What makes MemoryOS different

Capability	Filing Cabinets (Confluence/Notion)	MemoryOS
Passive indexing	Requires manual entry	Fully automatic
Captures the "why"	Stores documents only	Extracts decision rationale via LLM
Cross-system links	Siloed per tool	Unified graph across all sources
AI queryable	No MCP/agent interface	Native MCP server
Proactive context	Search only	Surfaces relevant facts unprompted
Self-correcting	Static	Reflection engine detects knowledge gaps
One-line install	Enterprise procurement	curl bash

2. SYSTEM ARCHITECTURE

2.1 Data Flow

MemoryOS operates as a four-layer pipeline. Data flows from source systems through an ingestion pipeline into a semantic knowledge graph, then is exposed to AI clients via a standardized MCP server.



2.2 Knowledge Graph Schema

MemoryOS uses a Property Graph stored in Neo4j. Unlike relational databases, relationships are first-class objects with types and properties, enabling multi-hop queries that trace causal chains across systems.

Node Types

Node	Key Properties	Purpose
Person	name, email, role, context_expertise[], tenure_days	Team members — deduped across all source systems
Artifact	title, body, artifact_type, source_system, external_id, url	Any content unit: message, PR, ticket, doc, commit
Decision	title, rationale, tradeoffs_made[], status	Captured decisions with explicit "why" and tradeoffs
Context	summary, key_themes[], source_artifact_id	LLM-extracted semantic summaries of artifact content

Relationship Types (Edges)

Relationship	From → To	Meaning
AUTHORED	Person → Artifact	This person created this content
STAKEHOLDER_IN	Person → Decision	This person had context on this decision
RATIONALE_FOR	Decision → Artifact/Context	This decision is explained by this evidence
LINKED_TO	Artifact → Artifact	Cross-system reference (e.g., Slack mentions PROJ-123)
REPLACES	Artifact → Artifact	Evolution chain — newer solution supersedes old
EXTRACTED_FROM	Context → Artifact	This summary was extracted from this source
PART_OF	Artifact → Artifact	Thread/comment hierarchy (reply → parent message)
SAME_AS	Artifact → Artifact	Deduplication — same document in two systems

2.3 MCP Server & Exposed Tools

The MCP (Model Context Protocol) server is the AI interface layer. It implements Anthropic's open standard for AI tool integration, meaning any MCP-compatible client can auto-discover and call MemoryOS tools without custom integration code.

`get_decision_backstory`

Parameters: `artifact_id`, `include_persons`, `max_depth`

Fetches the complete conversational backstory behind a Jira ticket or GitHub PR. Traverses the knowledge graph up to `max_depth` hops to surface related Slack threads, email discussions, and decision nodes. Returns a Claude-synthesized narrative explaining WHY the artifact exists.

`trace_rationale`

Parameters: `decision_query`, `time_range_days`, `focus_on`

Follows the complete reasoning path of an architecture or technical decision. Uses semantic similarity to find matching decisions, then traces the evidence chain backward through the graph. Returns stakeholders, alternatives considered, and tradeoffs made.

`surface_proactive_context`

Parameters: `intent`, `context_type`, `top_k`, `find_experts`

The "what should I know before I start?" tool. Given a work intent, surfaces semantically similar past artifacts, relevant prior decisions, and subject-matter experts. Returns a prioritized briefing.

`who_knows_about`

Parameters: `topic`, `active_within_days`

Identifies which team members have the most context on a specific topic. Ranks contributors by authored artifacts in the relevant semantic neighborhood.

`find_similar_decisions`

Parameters: `problem_description`, `include_superseded`

Searches for past decisions that addressed a similar problem. Prevents teams from reinventing solved problems. Returns prior solutions with rationale and current status.

3. INSTALLATION

3.1 Prerequisites

Requirement	Min Version	Notes
Node.js	18.0.0	Verify: <code>node --version</code>
npm	9.0.0	Included with Node.js
Docker Desktop	4.0+	Must be running before install
Git	Any	For source clone
Anthropic API Key	Any tier	Required for LLM extraction and tool responses
OpenAI API Key	Optional	For production embeddings. Use provider: local for dev/testing.

3.2 One-Line Installer (Recommended)

```
# Run this in your terminal
curl -fsSL https://raw.githubusercontent.com/dcash10181-gh/memoryos/main/install.sh
| bash
```

The installer automatically:

- Verifies Node.js 18+, npm, and Docker are present and running
- Clones the repository to `~/.memoryos/core`
- Runs `npm install` and `npm run build`
- Starts Neo4j in Docker with a secure auto-generated password
- Creates `~/.memoryos/config/config.yaml` with all options pre-populated
- Installs the memoryos CLI binary to `/usr/local/bin`
- Registers the daemon with `launchd` (macOS) or `systemd` (Linux) for auto-start

3.3 Manual Install

```
# Manual installation
git clone https://github.com/dcash10181-gh/memoryos.git
cd memoryos
npm install
```

```
npm run build

# Start Neo4j
export NEO4J_PASSWORD=your_secure_password
docker compose up -d neo4j

# Wait ~30 seconds for Neo4j, then:
node dist/index.js start
```


4. CONFIGURATION

4.1 config.yaml Reference

The main configuration file lives at `~/.memoryos/config/config.yaml`. All values support environment variable interpolation using `${VAR_NAME}` syntax.

```
# ~/.memoryos/config/config.yaml
version: "1.0"

daemon:
  port: 7890 # MCP server port
  log_level: info # debug | info | warn | error
  sync_interval: 300 # seconds between incremental syncs

graph:
  uri: bolt://localhost:7687
  user: neo4j
  password: ${NEO4J_PASSWORD}

embedding:
  provider: openai # openai | voyage | local
  model: text-embedding-3-small
  api_key: ${OPENAI_API_KEY}

llm:
  provider: anthropic
  model: claude-sonnet-4-20250514
  api_key: ${ANTHROPIC_API_KEY}

connectors:
  slack:
    enabled: true
    bot_token: ${SLACK_BOT_TOKEN}
    lookback_days: 90
  github:
    enabled: true
```

```
token: ${GITHUB_TOKEN}
org: your-org
repos: [] # empty = all accessible repos
```

4.2 Environment Variables

Variable	Required	Description
ANTHROPIC_API_KEY	Yes	Claude API key for LLM extraction and tool responses
NEO4J_PASSWORD	Yes	Neo4j database password (auto-generated by installer)
OPENAI_API_KEY	Recommended	For production-quality embeddings. Omit to use local fallback.
SLACK_BOT_TOKEN	If Slack enabled	Bot OAuth token (xoxb-...) from Slack App settings
SLACK_APP_TOKEN	If Slack enabled	App-level token (xapp-...) for Socket Mode
GITHUB_TOKEN	If GitHub enabled	Personal access token with repo read scope
JIRA_EMAIL	If Jira enabled	Atlassian account email address
JIRA_TOKEN	If Jira enabled	Jira API token from id.atlassian.com/manage-profile/security
CONFLUENCE_TOKEN	If Confluence	Same Atlassian API token as Jira
NOTION_TOKEN	If Notion enabled	Internal integration token from notion.so/my-integrations

■ **Tip:** Store all secrets in `~/.memoryos/config/.env` — never commit this file to git. The `.gitignore` in the repo already excludes it.

5. CONNECTORS

5.1 Slack

Tier 1 — High-Signal Conversational

Slack is the highest-value connector because it captures the informal, real-time conversations where most institutional knowledge actually lives. MemoryOS indexes channel messages and — critically — full thread replies, which contain the decision rationale that never makes it into tickets or docs.

Setup steps:

1. Create a Slack App at api.slack.com/apps
2. Add OAuth scopes: channels:read, channels:history, groups:history, users:read, users:read.email
3. Install app to workspace and copy the Bot OAuth Token (xoxb-...)
4. Enable Socket Mode and create an App-Level Token (xapp-...) with connections:write scope
5. Invite the bot to each channel: `/invite @your-bot-name`

5.2 GitHub

Tier 2 — Decision Tracking

GitHub captures the most structured decision artifacts: PR descriptions (the "why we changed this"), review comments (the debate), issues (the problem statement), and commit messages (compressed decisions). PRs with long review threads are the richest source of architectural rationale in most engineering orgs.

Setup steps:

1. Go to github.com → Settings → Developer settings → Personal access tokens → Tokens (classic)
2. Generate a new token with repo scope (read access to code, issues, and PRs)
3. Set org: your-github-org in `config.yaml`
4. Set repos: `[]` to index all accessible repos, or list specific ones

5.3 Jira

Tier 2 — Decision Tracking

Jira tickets capture the formal record of work but the real value is in the comments — where teams debate approach, surface blockers, and record tradeoffs. MemoryOS indexes both ticket bodies and all comment threads, linking them to related Slack discussions via cross-reference detection.

Setup steps:

1. Go to id.atlassian.com/manage-profile/security → Create and manage API tokens
2. Set host: <https://your-org.atlassian.net> in config.yaml
3. Set projects: [] to index all projects or list specific project keys

■ **Note:** Confluence and Notion connectors are available in Beta. Enable them in config.yaml once your Tier 1 and Tier 2 connectors are verified working.

6. TESTING PROCEDURES

6.1 Smoke Test Checklist

Run through this checklist in order after every install or major configuration change.

#	Check	Command	Expected
1	Neo4j container running	<code>docker ps grep memoryos-neo4j</code>	Should show "Up" status
2	Neo4j accepting connections	<code>curl -s http://localhost:7474 head -c 50</code>	Should return JSON with <code>bolt_routing</code> key
3	Node build successful	<code>ls dist/index.js dist/mcp/server.js</code>	Both files should exist
4	Config loads without error	<code>node dist/cli/index.js status</code>	Should print green checkmarks
5	Connector credentials valid	<code>node dist/index.js sync --full 2>&1 head -20</code>	Should show "[connector] Connected"
6	Events ingested	<code>node dist/index.js sync --full</code>	Watch for "[pipeline] Progress: N events"
7	MCP server starts	<code>node dist/index.js start</code>	Should print "MCP server running on :7890"
8	Health endpoint responds	<code>curl http://localhost:7890/health</code>	Should return <code>{"status":"ok"}</code>
9	Tools discoverable	<code>curl http://localhost:7890/tools</code>	Should return JSON with 5 tools listed

6.2 Neo4j Browser Verification

Open `http://localhost:7474` in your browser. Login: `neo4j / your NEO4J_PASSWORD`. Run these Cypher queries to verify ingestion:

Count all nodes by label

```
MATCH (n)
RETURN labels(n)[0] AS label, count(n) AS count
ORDER BY count DESC
```

Sample ingested artifacts

```
MATCH (a:Artifact)
RETURN a.title, a.source_system, a.artifact_type, a.created_at
LIMIT 20
```

All Person nodes

```
MATCH (p:Person)
RETURN p.name, p.email, p.source_system
ORDER BY p.name
```

Visualize a decision chain

```
MATCH path = (d:Decision)-[:RATIONALE_FOR|EXTRACTED_FROM*1..3]->(n)
RETURN path LIMIT 10
```

Most connected artifacts

```
MATCH (a:Artifact)-[r]-()
WITH a, count(r) AS degree
RETURN a.title, a.source_system, degree
ORDER BY degree DESC LIMIT 10
```

Open knowledge gaps

```
MATCH (g:KnowledgeGap {resolved: false})
RETURN g.type, g.description, g.confidence
ORDER BY g.confidence DESC
```

6.3 MCP Tool Testing via curl

Test each MCP tool directly with curl before connecting an AI client. This isolates tool logic from client behavior.

List available tools

```
curl http://localhost:7890/tools
```

Test get_decision_backstory

```
curl -X POST http://localhost:7890/mcp \
-H "Content-Type: application/json" \
-d '{"jsonrpc":"2.0","method":"tools/call",
"params":{"name":"get_decision_backstory",
"arguments":{"artifact_id":"YOUR-JIRA-123"}},
"id":1}'
```

Test surface_proactive_context

```
curl -X POST http://localhost:7890/mcp \
-H "Content-Type: application/json" \
-d '{"jsonrpc":"2.0","method":"tools/call",
"params":{"name":"surface_proactive_context",
"arguments":{"intent":"reviewing the auth service"}},
"id":2}'
```

6.4 Claude Desktop Integration Test

Add the following to `~/Library/Application Support/Claude/claude_desktop_config.json` and restart Claude Desktop:

```
# claude_desktop_config.json
{
  "mcpServers": {
    "memoryos": {
      "url": "http://localhost:7890/mcp"
    }
  }
}
```

```
}
```

Test prompts to verify the integration is working:

- "Use MemoryOS to tell me what decisions were made in this codebase"
- "Who on the team knows the most about the authentication system?"
- "Has this problem been solved before? [describe your current problem]"
- "What should I know before reviewing this PR?"
- "Why did we choose Neo4j over a relational database?"

7. CLI REFERENCE

Command	Description
<code>memoryos start</code>	Start the daemon and MCP server. Runs full sync on first launch.
<code>memoryos sync</code>	Run an incremental sync (only new content since last sync)
<code>memoryos sync --full</code>	Force a complete historical re-index of all enabled connectors
<code>memoryos ask ""</code>	Query institutional memory directly from the CLI
<code>memoryos gaps</code>	Show knowledge graph gaps found by the reflection engine
<code>memoryos status</code>	Health check — Neo4j connectivity and MCP server status

8. TROUBLESHOOTING

Neo4j fails to start

Debug command

```
docker logs memoryos-neo4j --tail 30
```

- Ensure Docker Desktop is running before starting MemoryOS
- Check ports 7474 and 7687 are not in use: `lsof -i :7474`
- Verify NEO4J_PASSWORD is set in `~/.memoryos/config/.env`
- On first launch Neo4j takes ~30 seconds to initialize — wait before retrying

Connector shows "Validation failed"

Debug command

```
node dist/index.js start 2>&1 | grep Validation
```

- Verify the token/key is correctly set in `~/.memoryos/config/.env`
- Check that env var names in config.yaml exactly match .env file (case-sensitive)
- For Slack: ensure the bot has been invited to the channels (/invite @bot-name)
- For GitHub: verify the PAT has repo scope and has not expired

Zero nodes in Neo4j after sync

Debug command

```
node dist/index.js sync --full 2>&1 | grep -E "error|0 events"
```

- Check that at least one connector is set to enabled: true in config.yaml
- Verify ANTHROPIC_API_KEY is set — ingestion pipeline requires Anthropic access
- Check lookback_days is large enough to capture historical content
- Run with LOG_LEVEL=debug for verbose output

MCP server not reachable from Claude Desktop

Debug command

```
curl http://localhost:7890/health
```

- Ensure memoryos start is running in a terminal window
- Verify port in claude_desktop_config.json matches daemon.port in config.yaml (default 7890)

- Restart Claude Desktop after editing the MCP config file
- Check macOS firewall is not blocking localhost connections

9. TECHNICAL CONCEPTS EXPLAINED

Plain-language explanations of the core technical concepts behind MemoryOS, useful for onboarding, documentation, and interview preparation.

Knowledge Graph vs. Vector Database

A vector database finds content that is semantically similar to a query — it answers "what is most like this?"

A knowledge graph stores typed relationships between entities and answers "how did X lead to Y?"

MemoryOS uses both: vector embeddings for similarity search, and a property graph for causal traversal.

The graph enables multi-hop reasoning — tracing a Slack thread through a decision to the PR that implemented it.

Cosine Similarity

Every piece of text is converted into a high-dimensional vector (list of numbers) called an embedding. These numbers encode the semantic meaning of the text. Cosine similarity measures the angle between two vectors — near zero means the texts mean nearly the same thing. MemoryOS uses a threshold of 0.88 for entity deduplication: two Person nodes with embeddings that are 88%+ similar are likely the same person across different systems.

Levenshtein Distance

A string comparison algorithm that counts the minimum single-character edits needed to transform one string into another. "Duane C." to "Duane Cash" is a small distance; "Duane Cash" to "John Smith" is large. MemoryOS combines this with cosine similarity — both signals must pass their thresholds to trigger a merge, creating a two-factor confidence check that prevents incorrect merges.

Cross-System Entity Deduplication

The same person appears as "Duane Cash" in Slack, "dcash10181" in GitHub, and "Duane C." in Jira. Without deduplication, the knowledge graph fragments into disconnected islands. MemoryOS runs a post-ingestion pass that merges Person nodes matching on email (exact) or name similarity + embedding similarity (soft). Artifacts get SAME_AS edges when URLs or content overlap across sources.

Model Context Protocol (MCP)

MCP is Anthropic's open standard for AI tool integration — analogous to USB for hardware. Before MCP, every AI integration required custom code per client. With MCP, a server publishes capabilities via a standardized JSON-RPC 2.0 interface, and any MCP-compatible client — Claude Desktop, GitHub Copilot, Cursor — can discover and invoke those capabilities automatically. MemoryOS implements an MCP server, making its knowledge tools available to any present or future AI client without additional integration work.

Reflection-Based Gap Detection

After each sync, MemoryOS self-audits the knowledge graph looking for structural anomalies: decisions with no supporting artifacts, Jira IDs mentioned in Slack but not linked in the graph, high-activity tickets with no captured rationale. Each gap gets a confidence score, and gaps above the threshold trigger a Claude-synthesized action plan. This is an application of agentic reflection — the system evaluates its own outputs and identifies where it is incomplete.

Stateful MCP Sessions

Unlike REST APIs that treat each request independently, the MemoryOS MCP server maintains session state across multiple tool calls. An AI client can call `surface_proactive_context`, then follow up with `get_decision_backstory` on a specific result, and the server remembers the conversation context. Sessions are managed per SSE connection and cleaned up automatically when the client disconnects.

10. ROADMAP

v1.1 — Connector Expansion

Q3 2025

- Linear (modern Jira alternative, popular at startups)
- Figma comments (design decision capture)
- Google Drive / Google Docs
- Microsoft Teams + SharePoint

v1.2 — Intelligence Layer

Q4 2025

- Automatic Architecture Decision Record (ADR) generation from PR + Slack data
- Onboarding accelerator — personalized context briefing for new hires by role/team
- Meeting transcript connector (Otter.ai, Fireflies, Zoom)
- Attrition risk scoring — surfaces knowledge concentration risk before an employee leaves

v2.0 — Enterprise Edition

2026

- Multi-tenant architecture for SaaS deployment
- RBAC — role-based access control at the graph level
- SSO / SAML integration for enterprise authentication
- Analytics dashboard — knowledge graph health metrics, gap trends, expert mapping

MemoryOS v1.0.0

Authored by Duane Cash

[github.com/dcash10181-gh/
memoryos](https://github.com/dcash10181-gh/memoryos)